Assignment 5.

1. Create a reducer function in a new file called playlistReducer.js that manages an array of songs, handling 'ADD_SONG' and 'REMOVE_SONG' actions using a switch case.

2. Build a simple Spotify-style playlist manager using useReducer in React — display a list of songs, and allow the user to add a new song by typing a name and clicking an 'Add' button.

3. Refactor your playlist manager so that each song in the list has a 'Delete' button next to it; dispatch a 'REMOVE_SONG' action from the Delete button in the child SongItem component.

4. Extend your reducer to handle an 'EDIT_SONG' action: allow users to click an 'Edit' button on a song, change its name in an input box, and save it by dispatching from the child component.

Hint: Pass the dispatch function as a prop to the child component that handles editing.

Answer:-

1 question :-

PlaylistReducer.js

```
const playlistReducer = (state, action) => {
  switch (action.type) {
    case "ADD_SONG":
      return [
        ...state,
        {
          id: Date.now(),
          name: action.payload,
        },
      ];

    case "REMOVE_SONG":
      return state.filter(
        (song) => song.id !== action.payload
      );

    case "EDIT_SONG":
      return state.map((song) =>
        song.id === action.payload.id
          ? {
              ...song,
              name: action.payload.name,
            }
          : song
      );
```

```
    default:
      return state;
  }
};

export default playlistReducer;
```

Question2.:- PlatlistManager.js

```
import React, { useReducer, useState } from "react";

import playlistReducer from "./playlistReducer";
import SongItem from "./SongItem";

const PlaylistManager = () => {
  const [songName, setSongName] = useState("");

  const [songs, dispatch] = useReducer(playlistReducer, [
    {
      id: 1,
      name: "Shape of You",
    },
    {
      id: 2,
      name: "Perfect",
    },
  ]);

  const handleAddSong = () => {
    if (!songName.trim()) return;

    dispatch({
      type: "ADD_SONG",
      payload: songName,
    });

    setSongName("");
  };

  return (
    <div>
      <h1>🎵 Spotify Playlist Manager</h1>

      <input
        type="text"
        placeholder="Enter Song Name"
```

```
        value={songName}
        onChange={(e) => setSongName(e.target.value)}
      />

      <button onClick={handleAddSong}>Add Song</button>

      <hr />

      {songs.map((song) => (
        <SongItem key={song.id} song={song} dispatch={dispatch} />
      ))}
    </div>
  );
};

export default PlaylistManager;
```

Question 3, 4:-

SongItem.jsx

```
import React, { useState } from "react";

const SongItem = ({ song, dispatch }) => {
  const [isEditing, setIsEditing] = useState(false);

  const [newName, setNewName] = useState(song.name);

  const handleSave = () => {
    dispatch({
      type: "EDIT_SONG",
      payload: {
        id: song.id,
        name: newName,
      },
    });

    setIsEditing(false);
  };

  return (
    <div
      style={{
        margin: "10px 0",
      }}
    >
```

```
      {isEditing ? (
        <>
          <input
            type="text"
            value={newName}
            onChange={(e) => setNewName(e.target.value)}
          />

          <button onClick={handleSave}>Save</button>
        </>
      ) : (
        <>
          <span>{song.name}</span>

          <button onClick={() => setIsEditing(true)}>Edit</button>

          <button
            onClick={() =>
              dispatch({
                type: "REMOVE_SONG",
                payload: song.id,
              })
            }
          >
            Delete
          </button>
        </>
      )}
    </div>
  );
};

export default SongItem;
```